7 Inheritance

Chapter Objectives

7.1 Base class access control

7.2 Using protected members

7.3 Constructors, destructors, and inheritance

7.4 Multiple inheritance

7.5 Virtual base classes

You were introduced to the concept of inheritance earlier in this book. Now it is time to explore it more thoroughly. Inheritance is one of the three principles of OOP and, as such, it is an important feature of C++. Inheritance does more than just support the concept of hierarchical classification; in Chapter-10- you will learn how inheritance provides support for polymorphism, another principal feature of OOP.
The topics covered in this chapter include base class access control and the **protected** access specifier, inheriting multiple base classes, passing arguments to base class constructors, and virtual base classes.

Review Skills Check

Before proceeding, you should be able to correctly answer the following questions and do the exercises.

1. When an operator is overloaded, does it lose any of its original functionality?
2. Must an operator be overloaded relative to a user-defined type, such as a class?
3. Can the precedence of an overloaded operator be changed? Can the number of operands be altered?
4. Given the following partially completed program, fill in the needed operator functions:

```
#include <iostream>
using namespace std;

class array
{
    int nums[10];
public:
    array();
    void set(int n[10]);
    void show();
    array operator+(array ob2);
    array operator-(array ob2);
    array operator==(array ob2);
};

array::array()
{
    int i;
    for(i=0; i<10; i++)
        nums[i] = 0;
}

void array::set(int *n)
{
    int i;

    for(i=0; i<10; i++)
        nums[i] = n[i];
}

void array::show()
```

```
{
    int i;

    for(i=0; i<10; i++)
        cout << nums[i] << ' ';

    cout << "\n";
}

// Fill in operator functions.

int main()
{
    array o1, o2, o3;

    int i[10] = {1, 2, 3, 4, 5, 6, 7, 8 ,9 ,10 };

    o1.set(i);
    o2.set(i);

    o3 = o1 + o2;
    o3.show();

    o3 = o1 - o3;
    o3.show();

    if(o1==o2)
        cout << "o1 equals o2\n";
    else
        cout << "o1 does not equal o2\n";

    if(o1==o3)
        cout << "o1 equals o3\n";
    else
        cout << "o1 does not equal o3\n";

    return 0;
}
```

Have the overloaded + add each element of each operand. Have the overloaded - subtract each element of the right operand from the left. Have the overloaded == return true if each element of each operand is the same and return false otherwise.

5. Convert the solution from Exercise 4 so it overloads the operators by using friend functions.

6. Using the class and support functions from Exercise 4, overload the ++ operator by using a member function and overload the – operator by using a friend. (Overload only the prefix forms of ++ and –.)

7. Can the assignment operator be overloaded by using a friend function?

7.1 BASE CLASS ACCESS CONTROL

When one class inherits another, it uses this general form:

```
class derived_class_name:access base_class_name
{
    // ...
}
```

Here *access* is one of three keywords: **public**, **private**, or **protected**. A discussion of the **protected** access specifier is deferred until the next section of this chapter. The other two are discussed here.

The access specifier determines how elements of the base class are inherited by the derived class. When the access specifier for the inherited base class is **public**, all public members of the base become public members of the derived class. If the access specifier is **private**, all public members of the base class become private members of the derived class. In either case, any private members of the base remain private to it and are inaccessible by the derived class.

It is important to understand that if the access specifier is **private**, public members of the base become private members of the derived class, but these members are still accessible by member functions of the derived class.

Technically, *access* is optional. If the specifier is not present, it is **private** by default if the derived class is a **class**. If the derived class is a **struct**, **public** is the default in the absence of an explicit access specifier. Frankly, most programmers explicitly specify *access* for the sake of clarity.

EXAMPLES

1. Here is a short base class and a derived class that inherits it (as public):

```
#include <iostream>
using namespace std;

class base
{
    int x;
public:
    void setx(int n) { x = n; }
    void showx() { cout << x << '\n'; }
};

// Inherit as public.
class derived : public base
{
    int y;
public:
    void sety(int n) { y = n; }
    void showy() { cout << y << '\n'; }
};

int main()
{
    derived ob;

    ob.setx(10); // access member of base class
```

```
    ob.sety(20); // access member of derived class

    ob.showx(); // access member of base class
    ob.showy(); // access member of derived class

    return 0;
}
```

As this program illustrates, because **base** is inherited as public, the public members of **base-setx()** and **showx()**-become public members of **derived** and are, therefore, accessible by any other part of the program. Specifically, they are legally called within **main()**.

2. It is important to understand that just because a derived class inherits a base as public, it does not mean that the derived class has access to the base's private members. For example, this addition to **derived** from the preceding example is incorrect:

```
class base
{
    int x;
public:
    void setx(int n) { x = n; }
    void showx() { cout << x << '\n'; }
};

// Inherit as public - this has an error!
class derived : public base
{
    int y;
public:
    void sety(int n) { y = n; }
    /*
       Cannot access private member of base class.
       x is a private member of base and not available
       within derived.
    */
    void show_sum() { cout << x+y << '\n'; } // Error!

    void showy() { cout << y << '\n'; }
};
```

In this example, the **derived** class attempts to access **x**, which is a private member of **base**. This is an error because the private parts of a base class remain private to it *no matter how it is inherited*.

3. Here is a variation of the program shown in Example 1; this time **derived** inherits **base** as private. This change causes the program to be in error, as indicated in the comments.

```
// This program contains an error.
#include <iostream>
using namespace std;

class base
{
```

```
    int x;
public:
    void setx(int n) { x = n; }
    void showx() { cout << x << '\n'; }
};

// Inherit base as private.
class derived : private base
{
    int y;
public:
    void sety(int n) { y = n; }
    void showy() { cout << y << '\n'; }
};

int main()
{
    derived ob;

    ob.setx(10); // ERROR - now private to derived class
    ob.sety(20); // access member of derived class - OK

    ob.showx(); // ERROR - now private to derived class
    ob.showy(); // access member of derived class - OK

    return 0;
}
```

As the comments in this (incorrect) program illustrate, both **showx()** and **setx()** become private to **derived** and are not accessible outside of it.

Keep in mind that **showx()** and **setx()** are still public within **base** no matter how they are inherited by some derived class. This means that an object of type **base** could access these functions anywhere. However, relative to objects of type **derived**, they become private. For example, given this fragment:

```
base base_ob;

base_ob.setx(1); // is legal because base_ob is of type base
```

the call to **setx()** is legal because **setx()** is public within **base**.

4. As stated, even though public members of a base class become private members of a derived class when inherited using the **private** specifier, they are still accessible *within* the derived class. For example, here is a *fixed* version of the preceding program:

```
// This program is fixed.
#include <iostream>
using namespace std;

class base
{
    int x;
public:
```

```
    void setx(int n) { x = n; }
    void showx() { cout << x << '\n'; }
};

// Inherit base as private.
class derived : private base
{
    int y;
public:
    // setx is accessible from within derived
    void setxy(int n, int m) { setx(n); y = m; }
    // show is accessible from within derived
    void showxy() { showx(); cout << y << '\n'; }
};

int main()
{
    derived ob;

    ob.setxy(10, 20);

    ob.showxy();

    return 0;
}
```

In this case, the functions **setx()** and **showx()** are accessed inside the derived class, which is perfectly legal because they are private members of that class.

EXERCISES

1. Examine this skeleton:

```
#include <iostream>
using namespace std;

class mybase
{
    int a, b;
public:
    int c;
    void setab(int i, int j) { a = i; b = j; }
    void getab(int &i, int &j) { i = a; j = b; }
};

class derived1 : public mybase
{
    // ...
};

class derived2 : private mybase
{
```

```
    // ...
};

int main()
{
    derived1 o1;
    derived2 o2;
    int i, j;

    // ...
}
```

Within **main()**, which of the following statements are legal?

```
A. o1.getab(i, j);

B. o2.getab(i, j);

C. o1.c = 10;

D. o2.c = 10;
```

2. What happens when a public member is inherited as public? What happens when it is inherited as private?

3. If you have not done so, try all the examples presented in this section. On your own, try various changes relative to the access specifiers and observe the results.

7.2 USING PROTECTED MEMBERS

As you know from the preceding section, a derived class does not have access to the private members of the base class. This means that if the derived class needs access to some member of the base, that member must be public. However, there will be times when you want to keep a member of a base class private but still allow a derived class access to it. To accomplish this goal, C++ includes the **protected** access specifier.

The **protected** access specifier is equivalent to the **private** specifier with the sole exception that protected members of a base class are accessible to members of any class derived from that base. Outside the base or derived classes, protected members are not accessible.

The **protected** access specifier can occur anywhere in the class declaration, although typically it occurs after the (default) private members are declared and before the public members. The full general form of a class declaration is shown here:

```
class class_name
{
    // private members
protected: // optional
    // protected members
public:
    // public members
};
```

When a protected member of a base class is inherited as public by the derived class, it becomes a protected member of the derived class. If the base is inherited as private, a protected member of the base becomes a private member of the derived class.
A base class can also be inherited as protected by a derived class. When this is the case, public and protected members of the base class become protected members of the derived class. (Of course, private members of the base class remain private to it and are not accessible by the derived class.)
The **protected** access specifier can also be used with structures.

EXAMPLES

1. This program illustrates how public, private, and protected members of a class can be accessed:

```
#include <iostream>
using namespace std;

class samp
{
    // private by default
    int a;
protected: // still private relative to samp
    int b;
public:
    int c;

    samp(int n, int m) { a = n; b = m; }
    int geta() { return a; }
    int getb() { return b; }
};

int main()
{
    samp ob(10, 10);

    // ob.b = 99; Error! b is protected and thus private
    ob.c = 30; // OK, c is public

    cout << ob.geta() << ' ';
    cout << ob.getb() << ' ' << ob.c << '\n';

    return 0;
}
```

   As you can see, the commented-out line is not permissible in **main()** because **b** is protected and is thus still private to **samp**.

2. The following program illustrates what occurs when protected members are inherited as public:

```
#include <iostream>
using namespace std;
```

```
class base
{
protected: // private to base
    int a, b; // but still accessible by derived
public:
    void setab(int n, int m) { a = n; b = m; }
};

class derived : public base
{
    int c;
public:
    void setc(int n) { c = n; }

    // this function has access to a and b from base
    void showbc()
    {
        cout << a << ' ' << b << ' ' << c << '\n';
    }
};

int main()
{
    derived ob;

    /*
       a and b are not accessible here because they are
       private to both base and derived.
    */

    ob.setab(1, 2);
    ob.setc(3);

    ob.showbc();

    return 0;
}
```

Because **a** and **b** are protected in **base** and inherited as public by **derived**, they are available for use by member functions of **derived**. However, outside of these two classes, **a** and **b** are effectively private and inaccessible.

3. As mentioned earlier, when a base class is inherited as protected, public and protected members of the base class become protected members of the derived class. For example, here the preceding program is changed slightly, inheriting **base** as protected instead of public:

```
// This program will not compile.
#include <iostream>
using namespace std;

class base
{
```

```
protected:     // private to base
    int a, b; // but still accessible by derived
public:
    void setab(int n, int m) { a = n; b = m; }
};

class derived : protected base // inherit as protected
{
    int c;
public:
    void setc(int n) { c = n; }

    // this function has access to a and b from base
    void showabc()
    {
        cout << a << ' ' << b << ' ' << c << '\n';
    }
};

int main()
{
    derived ob;

    // ERROR: setab() is now a protected member of base.
    ob.setab(1, 2); // setab() is not accessible here.

    ob.setc(3);

    ob.showabc();

    return 0;
}
```

As the comments now describe, because **base** is inherited as protected, its public and protected elements become protected members of **derived** and are therefore inaccessible within **main()**.

EXERCISES

1. What happens when a protected member is inherited as public? What happens when it is inherited as private? What happens when it is inherited as protected?

2. Explain why the protected category is needed?

3. In Exercise 1 from Section 7.1, if the **a** and **b** inside **myclass** were made into protected instead of private (by default) members, would any of your answers to that exercise change? If so, how?

7.3 CONSTRUCTORS, DESTRUCTORS, AND INHERITANCE

It is possible for the base class, the derived class, or both to have constructor and/or destructor functions. Several issues that relate to these situations are examined in this section.

When a base class and a derived class both have constructor and destructor functions, the constructor functions are executed in order of derivation. The destructor functions are executed in reverse order. That is, the base class constructor is executed before the constructor in the derived class. The reverse is true for destructor functions: the derived class's destructor is executed before the base class's destructor.

If you think about it, it makes sense that constructor functions are executed in order of derivation. Because a base class has no knowledge of any derived class, any initialization it performs is separate from and possibly prerequisite to any initialization performed by the derived class. Therefore, it must be executed first.

On the other hand, a derived class's destructor must be executed before the destructor of the base class because the base class underlies the derived class. If the base class's destructor were executed first, it would imply the destruction of of the derived class. Thus, the derived class's destructor must be called before the object goes out of existence.

So far, none of the preceding examples have passed arguments to either a derived or base class constructor. However, it is possible to do this. When only the derived class takes an initialization, arguments are passed to the derived class's constructor in the normal fashion. However, if you need to pass an argument to the constructor of the base class, a little more effort is needed. To accomplish this, a chain of argument passing is established. First, all necessary arguments to both the base class and the derived class are passed to the derived class's constructor. Using an expanded form of the derived class's constructor declaration, you then pass the appropriate arguments along to the base class. The syntax for passing along an argument from the derived class to the base class is shown here:

```
derived_constructor(arg_list): base(arg_list)
{
    // body of derived class constructor
}
```

Here *base* is the name of the base class. It is permissible for both the derived class and the base class to use the same argument.It is also possible for the derived class to ignore all arguments and just pass them along to the base.

EXAMPLES

1. Here is a very short program that illustrates when base class and derived class constructor and destructor functions are executed:

```
#include <iostream>
using namespace std;

class base
{
public:
    base() { cout << "Constructing base class\n"; }
    ~base() { cout << "Destructing base class\n"; }
};

class derived : public base
{
public:
    derived() { cout << "Constructing derived class\n"; }
    ~derived() { cout << "Destructing derived class\n"; }
};
```

```
int main()
{
    derived o;

    return 0;
}
```

This program displays the following output:

```
Constructing base class
Constructing derived class
Destructing derived class
Destructing base class
```

As you can see, the constructors are executed in order of derivation and the destructors are executed in reverse order.

2. This program shows how to pass an argument to a derived class's constructor:

```
#include <iostream>
using namespace std;

class base
{
public:
    base() { cout << "Constructing base class\n"; }
    ~base() { cout << "Destructing base class\n"; }
};

class derived : public base
{
    int j;
public:
    derived(int n)
    {
        cout << "Constructing derived class\n";
        j = n;
    }
    ~derived() { cout << "Destructing derived class\n"; }
    void showj() { cout << j << '\n'; }
};

int main()
{
    derived o(10);

    o.showj();

    return 0;
}
```

Notice that the argument is passed to the derived class's constructor in the normal fashion.

3. In the following example, both the derived class and the base class constructors take arguments. In this specific case, both use the same argument, and the derived class simply passes along the argument to the base.

```
#include <iostream>
using namespace std;

class base
{
    int i;
public:
    base(int n)
    {
        cout << "Constructing base class\n";
        i = n;
    }
    ~base() { cout << "Destructing base class\n"; }
    void showi() { cout << i << '\n'; }
};

class derived : public base
{
    int j;
public:
    derived(int n) : base(n) // pass arg to base class.
    {
        cout << "Constructing derived class\n";
        j = n;
    }
    ~derived() { cout << "Destructing derived class\n"; }
    void showj() { cout << j << '\n'; }
};

int main()
{
    derived o(10);

    o.showi();
    o.showj();

    return 0;
}
```

Pay special attention to the declaration of **derived**'s constructor. Notice how the parameter **n** (which receives the initialization argument) is both used by **derived()** and passed to **base()**.

4. In most cases, the constructor functions for the base and derived classes will *not* use the same argument. When this is the case and you need to pass one or more arguments to each, you must pass to the derived class's constructor *all* arguments needed by *both* the derived class and the base class. Then the derived class simply passes along to the base those arguments required by it. For example, this program shows how to pass an argument to the derived class's constructor and another one to the base class:

```
#include <iostream>
using namespace std;

class base
{
    int i;
public:
    base(int n)
    {
        cout << "Constructing base class\n";
        i = n;
    }
    ~base() { cout << "Destructing base class\n"; }
    void showi() { cout << i << '\n'; }
};

class derived : public base
{
    int j;
public:
    derived(int n, int m) : base(m) // pass arg to base class
    {
        cout << "Constructing derived class\n";
        j = n;
    }
    ~derived() { cout << "Destructing derived class\n"; }
    void showj() { cout << j << '\n'; }
};

int main()
{
    derived o(10, 20);

    o.showi();
    o.showj();

    return 0;
}
```

5. It is not necessary for the derived class' constructor to actually use an argument in order to pass one to the base class. If the derived class does not need an argument, it ignores the argument and simply passes it along. For example, in this fragment, parameter **n** is not used by **derived()**. Instead, it is simply passed to **base()**:

```
class base
{
    int i;
public:
    base(int n)
    {
        cout << "Constructing base class\n";
```

```
        i = n;
    }
    ~base() { cout << "Destructing base class\n"; }
    void showi() { cout << i << '\n'; }
};

class derived : public base
{
    int j;
public:
    derived(int n) : base(n) // pass arg to base class.
    {
        cout << "Constructing derived class\n";
        j = 0;
    }
    ~derived() { cout << "Destructing derived class\n"; }
    void showj() { cout << j << '\n'; }
};
```

EXERCISES

1. Given the following skeleton, fill in the constructor function for **myderived**. Have it pass along a pointer to an initialization string to **mybase**. Also, have **myderived()** initialize **len** to the length of the string.

```
#include <iostream>
#include <cstring>
using namespace std;

class mybase
{
    char str[80];
public:
    mybase(char *s); { strcpy(str, s); }
    char *get() { return str; }
};

class myderived : public mybase
{
    int len;
public:
    // add myderived() here
    int getlen() { return len; }
    void show() { cout << get() << '\n'; }
};

int main()
{
    myderived ob("hello");

    ob.show();
```

```
    cout << ob.getlen() << '\n';

    return 0;
}
```

2. Using the following skeleton, create appropriate **car()** and **truck()** constructor functions. Have each pass along appropriate arguments to **vehicle**. In addition, have **car()** initialize **passengers** as specified when object is created. Have **truck()** initialize **loadlimit** as specified when an object is created.

```
#include <iostream>
using namespace std;

// A base class  for various types of vehicle.
class vehicle
{
    int num_wheels;
    int range;
public:
    vehicle(int w, int r)
    {
        num_wheels = w;
        range = r;
    }
    void showv()
    {
        cout << "Wheels: " << num_wheels << '\n';
        cout << "Range: " << range << '\n';
    }
};

class car : public vehicle
{
    int passengers;
public:
    // insert car() constructor here
    void show()
    {
        showv();
        cout << "Passengers: " << passengers << '\n';
    }
};

class truck : public vehicle
{
    int loadlimit;
public:
    // insert truck() constructor here
    void show()
    {
        showv();
        cout << "Loadlimit: " << loadlimit << '\n';
```

```
    }
};

int main()
{
    car c(5, 4, 500);
    truck t(3000, 12, 1200);

    cout << "Car: \n";
    c.show();
    cout << "\nTruck:\n";
    t.show();

    return 0;
}
```

Have **car()** and **truck()** declare objects like this:

```
car ob(passengers, wheels, range);
truck(loadlimit, wheels, range);
```

7.4 MULTIPLE INHERITANCE

There are two ways that a derived class can inherit more than one base class. First, a derived class can be used as a base class for another derived class, creating a multilevel class hierarchy. In this case, the original base class is said to be an *indirect* base class of the second derived class. (Keep in mind that any class-no matter how it is created-can be used as a base class.) Second, a derived class can directly inherit more than one base class. In this situation, two or more base classes are combined to help create the derived class. There are several issues that arise when multiple base classes are involved, and these issues are examined in this section.
When a base class is used to derive a class that is used as a base class for another derived class, the constructor functions of all three classes are called in order of derivation. (This is a generalization of the principle you learned earlier in this chapter.) Also, destructor functions are called in reverse order. Thus, if class *B!* is inherited by *D1*, and *D1* is inherited by *D2*, *B1*'s constructor is called first, followed by *D1*'s, followed by *D2*'s. The destructors are called in reverse order.
When a derived class directly inherits multiple base classes, it uses this expanded declaration:

```
class derived_class_name : access base1, access base2, ...,
   access baseN
{
    // ... body of class
}
```

Here *base1* through *baseN* are the base class names and *access* is the access specifier, which can be different for each base class.When multiple base classes are inherited, constructors are executed in the order, left to right, that the base classes are specified. Destructors are executed in the opposite order.
When a class inherits multiple base classes that have constructors that require arguments, the derived class passes the necessary arguments to them by using this expanded form of the derived class' constructor function:

```
derived_constructor(arg_list) : base1(arg_list), base2(arg_list),
    ..., baseN(arg-list)
```

```
{
    // body of derived class constructor
}
```

Here *base1* through *baseN* are the names of the base classes.
When a derived class inherits a hierarchy of classes, each derived class in the chain must pass back to its preceding base any arguments it needs.

EXAMPLES

1. Here is an example of a derived class that inherits a class derived from another class. Notice how arguments are passed along the chain from **D2** to **B1**.

```
// Multiple Inheritance
#include <iostream>
using namespace std;

class B1
{
    int a;
public:
    B1(int x) { a = x; }
    int geta() { return a; }
};

// Inherit direct base class.
class D1 : public B1
{
    int b;
public:
    D1(int x, int y) : B1(y) // pass y to B1
    {
        b = x;
    }
    int getb() { return b; }
};

// Inherit a derived class and an indirect base.
class D2 : public D1
{
    int c;
public:
    D2(int x, int y, int z) : D1(y, z) // pass args to D1
    {
        c = x;
    }

    /*
       Because bases inherited as public, D2 has access
       to public elements of both B1 and D1.
    */
    void show()
    {
```

```
        cout << geta() << ' ' << getb() << ' ';
        cout << c << '\n';
    }
};

int main()
{
    D2 ob(1, 2, 3);

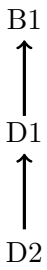
    ob.show();
    // geta() and getb() are still public here
    cout << ob.geta() << ' ' << ob.getb() << '\n';

    return 0;
}
```

The call to **ob.show()** displays **3 2 1**. In this example, **B1** is an indirect base class of **D2**. Notice that **D2** has access to the public members of both **D1** and **B1**. As you should remember, when public members of a base class are inherited as public, they become public members of the derived class. Therefore, when **D1** inherits **B1**, **geta()** becomes a public member of **D1**, which becomes a public member of **D2**.

As the program illustrates, each class in a class hierarchy must pass all arguments required by each preceding base class. Failure to do so will generate a compile-time error.

The class hierarchy created in this program is illustrated here:



Before we move on, a short discussion about how to draw C++-style inheritance graphs is in order. In the preceding graph, notice that the arrows point up instead of down. Traditionally, C++ programmers usually draw inheritance charts as directed graphs in which the arrow points from the derived class to the base class. While newcomers sometimes find this approach counter-intuitive, it is nevertheless the way inheritance charts are usually depicted in C++.

2. Here is a reworked version of the preceding program, in which a derived class directly inherits two base classes:

```
#include <iostream>
using namespace std;

// Create first base class.
class B1
{
    int a;
public:
```

```
    B1(int x) { a = x; }
    int geta() { return a; }
};


// Create second base class.
class B2
{
    int b;
public:
    B2(int x) { b = x; }
    int getb() { return b; }
};


// Directly inherit two base classes.
class D : public B1, public B2
{
    int c;
public:
    // here z and y are passed directly to B1 and B2
    D(int x, int y, int z) : B1(z), B2(y)
    {
        c = x;
    }

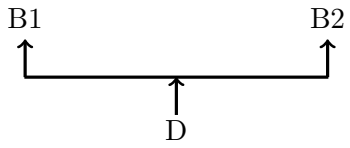
    /*
       Because bases inherited as public, D has access
       to public elements of both B1 and B2.
    */
    void show()
    {
        cout << geta() << ' ' << getb() << ' ';
        cout << c << '\n';
    }
};

int main()
{
    D ob(1, 2, 3);

    ob.show();

    return 0;
}
```

In this version, the arguments to **B1** and **B2** are passed individually to these classes by **D**. This program creates a class that looks like this:



3. The following program illustrates the order in which constructor and destructor functions are called when a derived class directly inherits multiple base classes:

```
#include <iostream>
using namespace std;

class B1
{
public:
    B1() { cout << "Constructing B1\n"; }
    ~B1() { cout << "Destructing B1\n"; }
};

class B2
{
public:
    B2() { cout << "Constructing B2\n"; }
    ~B2() { cout << "Destructing B2\n"; }
};

// Inherit two base classes.
class D : public B1, public B2
{
public:
    D() { cout << "Constructing D\n"; }
    ~D() { cout << "Destructing D\n"; }
};

int main()
{
    D ob;

    return 0;
}
```

This program displays the following:

```
Constructing B1
Constructing B2
Constructing D
Destructing D
Destructing B2
Destructing B1
```

As you have learned, when multiple direct base classes are inherited, constructors are called in order, left to right, as specified in the inheritance list. Destructors are called in reverse order.

EXERCISES

1. What does the following program display? (Try to determine this without actually running the program.)

```
#include <iostream>
using namespace std;

class A
{
public:
    A() { cout << "Constructing A\n"; }
    ~A() { cout << "Destructing A\n"; }
};

class B
{
public:
    B() { cout << "Constructing B\n"; }
    ~B() { cout << "Destructing B\n"; }
};

class C : public A, public B
{
public:
    C() { cout << "Constructing C\n"; }
    ~C() { cout << "Destructing C\n"; }
};

int main()
{
    C ob;

    return 0;
}
```

2. Using the following class hierarchy, create **C**'s constructor so that it initializes **k** and passes on argument to **A()** and **B()**.

```
#include <iostream>
using namespace std;

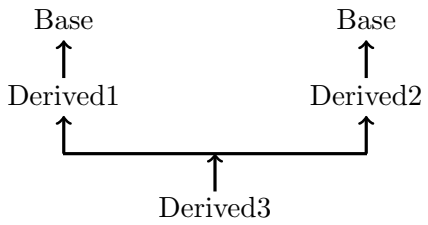
class A
{
    int i;
public:
    A(int a) { i = a; }
};
```

```
class B
{
    int j;
public:
    B(int a) { j = a; }
};

class C : public A, public B
{
    int k;
public:
    /*
       Create C() so that it initializes k
       and passes arguments to both A() and B()
    */
};
```

7.5 VIRTUAL BASE CLASSES

A potential problem exists when multiple base classes are directly inherited by a derived class. To understand what this problem is, consider the following class hierarchy:



Here the base class *Base* is inherited by both *Derived1* and *Derived2*. *Derived3* directly inherits both *Derived1* and *Derived2*. However, this implies that *Base* is actually inherited twice by *Derived3*-first it is inherited through *Derived1*, and then again through *Derived2*. This causes ambiguity when a member of *Base* is used by *Derived3*. Since two copies of *Base* are included in *Derived3*, is a reference to a member of *Base* referring to the *Base* inherited indirectly through *Derived1* or to the *Base* inherited indirectly through *Derived2*? To resolve this ambiguity. C+ includes a mechanism by which only one copy of *Base* will be included in *Derived3*. This feature is called a *virtual base class*.

In situations like the one just described, in which a derived class indirectly inherits the same base class more than once, it is possible to prevent two copies of the base from being present in the derived object by having that base class inherited as virtual by any derived classes. Doing this prevents two (or more) copies of the base from being present in any subsequent derived class that inherits the base class indirectly. The **virtual** keyword precedes the base class access specifier when it is inherited by a derived class.

EXAMPLES

1. Here is an example that uses a virtual base class to prevent two copies of **base** from being present in **derived3**.

```
// This program uses a virtual base class.
#include <iostream>
using namespace std;

class base
{
public:
    int i;
};

// Inherit base as virtual.
class derived1 : virtual public base
{
public:
    int j;
};

// Inherit base as virtual here, too.
class derived2 : virtual public base
{
public:
    int k;
};

/*
   Here, derived3 inherits both derived1 and derived2.
   However, only one copy of base is present.
*/
class derived3 : public derived1, public derived2
{
public:
    int product() {  return i * j * k; }
};

int main()
{
    derived3 ob;

    ob.i = 10; // unambiguous because only one copy present
    ob.j = 3;
    ob.k = 5;

    cout << "Product is " << ob.product() << '\n';

    return 0;
}
```

If **derived1** and **derived2** had not inherited **base** as virtual, the statement

```
ob.i = 10;
```

would have been ambiguous and a compile-time error would have resulted. (See Exercise 1, below.)

2. It is important to understand that when a base class is inherited as virtual by a derived class, that base class still exists within that derived class. For example, assuming the preceding program, this fragment is perfectly valid:

```
derived1 ob;
ob.i = 100;
```

The only difference between a normal base class and a virtual one occurs when an object inherits the base more than once. If virtual base classes are used, only one base class is present in the object. Otherwise, multiple copies will be found.

EXERCISES

1. Using the program in Example 1, remove the **virtual** keyword and try to compile the program. See what type of errors result.
2. Explain why a virtual base class might be necessary.

SKILLS CHECK

Mastery Skills Check

At this point you should be able to perform the following exercises and answer the questions.

1. Create a generic base class called **building** that stores the number of floors a building has, the number of rooms, and its total square footage. Create derived class called **house** that inherits **building** and also stores the number of bedrooms and the number of bathrooms. Next, create a derived class called **office** that inherits **building** and also stores the number of fire extinguishers and the number of telephones. Note: Your solution may differ from the answer given in the back of this book. However, if it is functionally the same, count it as correct.
2. When a base class is inherited as public by the derived class, what happens to its public members? What happens to its private members? If the base is inherited as private by the derived class, what happens to its public and private members?
3. Explain what **protected** means. (Be sure to explain what it means both when referring to members of a class and when it is used as an inheritance access specifier.)
4. When one class inherits another, when are the classes' constructors called? When are their destructors called?
5. Given this skeleton, fill in the details as indicated in the comments:

```
#include <iostream>
using namespace std;

class planet
{
protected:
    double distance; // miles from the sun
    int revolve; // in days
public:
    planet(double d, int r) { distance = d; revolve = r; }
};
```

```
class earth : public planet
{
    double circumference; // circumference of orbit
public:
    /*
       Create earth(double d, int r). Have it pass the
       distance and days of revolution back to planet.
       Have it compute the circumference of the orbit.
       (Hint: circumference = 2r*3.1416.)
    */
    /*
       Create a function called show() that displays the
       information.
    */
};

int main()
{
    earth ob(93000000, 365);

    ob.show();

    return 0;
}
```

6. Fix the following program:

```
/*
   A variation on the vehicle hierarchy. But
   this program contains an error. Fix it. Hind:
   try compiling it as is and observe the error
   messages.
*/
#include <iostream>
using namespace std;

// A base class for various types of vehicles.
class vehicle
{
    int num_wheels;
    int range;
public:
    vehicle(int w, int r)
    {
        num_wheels = w;
        range = r;
    }
    void showv()
    {
        cout << "Wheels: " << num_wheels << '\n';
        cout << "Range: " << range << '\n';
```

```
    }
};

enum motor {gas, electric, diesel};

class motorized : public vehicle
{
    enum motor mtr;
public:
    motorized(enum motor m, int w, int r) : vehicle(w, r)
    {
        mtr = m;
    }
    void showm()
    {
        cout << "Motor: ";
        switch(mtr)
        {
            case gas : cout << "Gas\n";
            break;
            case electric : cout << "Electric\n";
            break;
            case diesel : cout << "Diesel\n";
            break;
        }
    }
};

class road_use : public vehicle
{
    int passengers;
public:
    road_use(int p, int w, int r) : vehicle(w, r)
    {
        passengers = p;
    }
    void showr()
    {
        cout << "Passengers: " << passengers << '\n';
    }
};

enum steering { power, rack_pinion, manual };

class car : public motorized, public road_use
{
    enum steering strng;
public:
    car(enum steering s, enum motor m, int w, int r, int p) :
        road_use(p, w, r), motorized(m, w, r), vehicle(w, r)
    {
```

```
        strng = s;
    }
    void show()
    {
        showv();
        showr();
        showm();
        cout << "Steering: ";
        switch(strng)
        {
            case power : cout << "Power\n";
            break;
            case rack_pinion : cout << "Rack and Pinion\n";
            break;
            case manual : cout << "Manual\n";
            break;
        }
    }
};

int main()
{
    car c(power, gas, 4, 500, 5);

    c.show();

    return 0;
}
```

Cumulative Skills Check

This section checks how well you have integrated material in this chapter with that from the preceding chapters.

1. In Exercise 6 from the preceding Mastery Skills Check section, you might have seen a warning message (or perhaps an error message) concerning the use of the **switch** statement within **car** and **motorized**. Why?

2. As you know from the preceding chapter, most operators overloaded in a base class are available for use in a derived class. Which one or ones are not? Can you offer a reason why this is the case?

3. Following is a reworked version of the **coord** class from the previous chapter. This time it is used as a base for another class called **quad**, which also maintains the quadrant the specific point is in. On your own, run this program and try to understand its output.

```
/*
   Overload the +, -, and = relative to coord class. Then
   use coord as a base for quad.
*/
#include <iostream>
using namespace std;
```

```
class coord
{
public:
    int x, y; // coordinate values
    coord() { x=0; y=0; }
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    coord operator+(coord ob2);
    coord operator-(coord ob2);
    coord operator=(coord ob2);
};

// Overload + relative to coord class.
coord coord::operator+(coord ob2)
{
    coord temp;

    cout << "Using coord operator+()\n";

    temp.x = x + ob2.x;
    temp.y = y + ob2.y;

    return temp;
}

// Overload - relative to coord class.
coord coord::operator-(coord ob2)
{
    coord temp;

    cout << "Using coord operator-()\n";

    temp.x = x - ob2.x;
    temp.y = y - ob2.y;

    return temp;
}

// Overload = relative to coord.
coord coord::operator=(coord ob2)
{
    cout << "Using coord operator=()\n";

    x = ob2.x;
    y = ob2.y;

    return *this; // return the object that is assigned to
}

class quad : public coord
{
```

```
    int quadrant;
public:
    quad()
    {
        x = 0;
        y = 0;
        quadrant = 0;
    }
    quad(int x, int y) : coord(x, y)
    {
        if(x>=0 && y >= 0)
            quadrant = 1;
        else if(x<0 && y>=0)
            quadrant = 2;
        else if(x<0 && y <0)
            quadrant = 3;
        else
            quadrant = 4;
    }
    void showq()
    {
        cout << "Point in Quadrant: " << quadrant << '\n';
    }
    quad operator=(coord ob2);
};

quad quad::operator=(coord ob2)
{
    cout << "Using quad operator=()\n";

    x = ob2.x;
    y = ob2.y;
    if(x>=0 && y >= 0)
        quadrant = 1;
    else if(x<0 && y>=0)
        quadrant = 2;
    else if(x<0 && y <0)
        quadrant = 3;
    else
        quadrant = 4;

    return *this;
}

int main()
{
    quad o1(10, 10), o2(15, 3), o3;
    int x, y;

    o3 = o1 + o2; // add two objects - this calls operator+()
    o3.get_xy(x, y);
```

```
    o3.showq();
    cout << "(o1+o2) X: " << x << ", Y: " << y << "\n";

    o3 = o1 - o2; // subtract two objects
    o3.get_xy(x, y);
    o3.showq();
    cout << "(o1-o2) X: " << x << ", Y: " << y << "\n";

    o3 = o1; // assign an object
    o3.get_xy(x, y);
    o3.showq();
    cout << "(o3=o1) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

4. Again on your own, convert the program shown in Exercise 3 so that it uses friend operator functions.